

wire vs reg

Classic Verilog splits names into two families: nets, declared with wire, and variables, declared with reg

Nets versus variables

- A wire is a net
- You drive it with continuous assign
- A reg is a variable
- You drive it inside procedural blocks
- Input ports are nets in IEEE 1364
- Output can be a net if assign drives it, or a reg if always drives it

Browser lab

igital Design and Verification Platform

Home

Too

re / Tools / wire vs reg

Challenges (1/22)

Quiz: net. Classic continuous-assign LHS is a? Answer: wire

Your answer

Show hintCheckNextIdle

quiz-net

quiz-var

quiz-myth

quiz-input

starter

illegal-assign-reg

combo-reg

ff-preset

bad-wire-always

out-assign

out-always

input-wire

input-reg-bad

explain

toggle-type

quiz-output

quiz-av

scenario-bad

matrix-spot

out-assign-reg-bad

quiz-1364

full-path

Core ideas

wire (net)

Driven by `assign`, gates, or port links.

reg (variable)

LHS of procedural `=` / `<=` in `always`.

Myth

`reg` ≠ flip-flop. Combo `always` still uses `reg`.

Legality playground

Pick a drive style and declaration type — verdict updates live.

Drive: assignType: wire

wire y;
assign y = a & b;

LEGAL — continuous assign drives a net

Scenarios:

Combo via assign — Continuous AND — y is a net.

Combo via always — Same AND in always @(*) — y is still a reg.

Clocked FF — posedge procedural store — reg (and a real FF).

Illegal: assign to reg — Shows the classic error.

Illegal: always to wire — Procedural LHS must be a variable.

output + assign — Port kind follows the driver.

output + always — Procedural output — reg.

Preset legal assign+wire

Preset illegal assign+reg

Preset combo always+reg

Preset posedge+reg

Explain verdict

Drive + type matrix

drive / type	wire	reg
continuous assign	OK	NO
always @(*) combo	NO	OK
always @(posedge clk)	NO	OK
module input port	OK	NO
output + assign	OK	NO
output + always	NO	OK

(explain or change combo)

starter assign + wire

Cheat sheet

Driver	Needs
assign / gates	wire (net)
always / initial	reg (variable)
input port	net (not reg in classic)
output port	wire if assign, reg if always
SystemVerilog Logic	Often replaces both (later course)

• Synthesis cares about edge vs level sensitivity — not the keyword `reg`.

• Starter: legal wire+assign, illegal reg+assign.

Wire vs reg lab starter

Real Verilog practice

```
wsl - module03-wire-vs-reg/examples/navigation

# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/wire_reg/wire_vs_reg.v
// wire_vs_reg.v - assign-driven net vs always-driven variable
module wire_reg_demo(
    input  a,
    input  b,
    output y_net,
    output reg y_var
);
    assign y_net = a & b;
    always @(*) y_var = a & b;
endmodule

$ iverilog -t null examples/wire_reg/wire_vs_reg.v
(syntax check passed)
```

Pitfalls to watch

- Do not read reg as “register hardware”, combo always blocks still need a reg on the left
- Do not continuous-assign a reg or procedural-assign a wire
- SystemVerilog logic blurs the picture in later courses
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load the starter and finish a few challenges
- On real Verilog
- When you are ready